

RECURSION II

PRACTICE: (S7-S10)

```
def foo(N):  
    if N <= 1:  
        return 0  
    return 1 + foo(N // 2)  
  
def fizz(N):  
    if N == 0:  
        return 0  
    return N % 10 + fizz(N // 10)
```

What do these print?

- (S7) `print(foo(3))`
- (S8) `print(fizz(1100))`

PRACTICE:

We want to write the function `ping_pong(N)` which prints "ping" and then "pong" in alternating order for a total of N prints.

L11 Does this code work? Why or why not? (*Expected* outputs at right.)

```
def ping_pong(N):  
    if N <= 0:  
        return  
    if N % 2 == 1:  
        print("ping")  
    else:  
        print("pong")  
    ping_pong(N - 1)
```

`ping_pong(3)` prints: `ping`

`pong` , `ping`

`ping_pong(2)` prints: `ping` then `pong`

`ping_pong(1)` prints: `ping`

HELPER FUNCTIONS

Sometimes to do recursion we need to remember a bit more information than is provided to the overall problem.

In this case, what other information do we need for `ping_pong(N)` to get a working recursive solution?

Why can't we just recursively call `ping_pong(N-1)`?

PRACTICE C12

We want to write the function `ping_pong(N)` which prints "ping" and then "pong" in alternating order for a total of N prints.

```
def ping_pong_helper(N, should_ping):  
    # TODO: do something here  
  
def ping_pong(N):  
    # TODO: pick second argument  
    ping_pong_helper(N, _____)
```

<code>ping_pong(3)</code>	prints:	<code>ping</code>	<code>pong</code>	<code>ping</code>
<code>ping_pong(2)</code>	prints:	<code>ping</code>	then	<code>pong</code>
<code>ping_pong(1)</code>	prints:	<code>ping</code>		

PRACTICE:

We want to write a function `sum_numbers` that gets the sum of all integers in a list recursively. Answer these questions in **C12**, either in English or with code snippets.

1. What triggers the base case(s)? What should you return in that case?
2. What is the little bit of work done on each step? How do we combine information from this call with the result of the rest of the recursion?
3. What do we tell the function to do recursively? (List slicing may be helpful here.)

We'll take the answers above to build this solution...

```
def sum_numbers(nums_list):  
    # TODO
```

PRACTICE (S9, S10)

We'll take the answers above to build this solution...

```
def sum_numbers(nums_list):  
    if nums_list == []:  
        return ____S9____  
  
    first = nums_list[0] # the first element of the input list (an int)  
    rest = nums_list[1:] # the rest of the input list (a list of ints)  
    return _____S10_____
```

PRACTICE:

Consider we want to write the function `remove_spaces(word)` that takes in a string and returns the same string without any vowels in it.

So `remove_spaces("Hello")` returns `"Hlll"`

Before writing any code, think together then discuss:

- What is the base case? (Why is it the empty string? `" "`)
- What is the little bit of work done on each step? How do we combine information from this call with the result of the rest of the recursion?
- What does our recursive call look like?

PRACTICE: C14

Finish writing this function:

```
def remove_spaces(word):  
    # TODO: What do you put here?
```

PRACTICE

Consider we want to write the **recursive** function `find(lst, target)` that returns the position of `target` in the `lst` or `-1` if it's not present. We'll do a *linear search*, inspecting position 0, then 1, then 2, then 3, ... until we find what we're looking for or run out of positions!

Before writing any code:

- What is the base case(s)?
- What is the recursive case?
- What is the work done on each step?
- What additional information needs to be passed in to do a Linear Search/what's the signature of the helper function?

HELPER FUNCTIONS

Linear Search works by repeatedly shrinking the *feasible search area*.

- `idx` is the left-most/lowest index that might still hold the target
- want to keep searching until we find our target or `idx` is big enough that there's nothing left to search in the list!

PRACTICE: (C14)

Finish writing this function:

```
def find(lst: list[int], target: int) -> int:
    return find_helper(lst, target, _____)
def find_helper(lst: list[int], target: int, idx: int) -> int:
    # TODO!
```

PRACTICE: (L15)

Consider we want to write the function `find_factors(N)` that returns a set containing all positive factors of the input integer `N`.

A number `x` is a factor of `N` if and only if `N % x == 0`.

Before writing any code (L15)

- What is the base case?
- What is the recursive case?
- What is the work done on each step?

HELPER FUNCTIONS

Sometimes to do recursion we need to remember a bit more information than is provided to the overall problem.

In this case, what other information do we need for `find_factors(N)` to get a working recursive solution?

Why can't we just recursively call `find_factors(N-1)`?

PRACTICE: (C16)

Finish writing this function:

```
def find_factors_helper(current, N):  
    # TODO: What do you put here?  
    # need to return a set of all factors of N  
  
def find_factors(N):  
    return find_factors_helper(1, N)
```

```
def find_factors_helper(current, N):  
    if current > N // 2:  
        return set()  
  
    if N % current == 0:  
        return {current} | find_factors_helper(current + 1, N)  
    else:  
        return find_factors_helper(current + 1, N)  
  
def find_factors(N):  
    return find_factors_helper(1, N)
```